

Primer Parcial de Programación Orientada a Objetos (72.33)

19/09/2024

Ejercicio 1 (3 puntos)

Se desea modelar un **rango de meses** indicando el **mes y año de inicio** del rango y el **mes y año de fin** del rango. Ambos extremos del rango son **inclusive**. Para ello deberá utilizar la clase inmutable **java.time.YearMonth** que modela un mes de un año (ya existe, no tiene que implementarla). Para simplificar el ejercicio, se asume que la instancia de fin del rango no es anterior a la instancia de inicio del rango y no corresponde validarlo. Por defecto el salto del rango es de un mes pero se debe permitir instanciar al rango con un tercer **parámetro opcional** que corresponde a una **función** que indica **cómo conseguir la próxima instancia de YearMonth** a partir de la instancia actual de YearMonth.

Implementar YearMonthRange y todo lo necesario para que, con el siguiente programa de prueba, se obtenga la salida indicada en los comentarios:

```
public static void main(String[] args) {
    YearMonth now = YearMonth.of(2024, 9); // Ejemplos de uso de java.time.YearMonth
    System.out.println(now); // 2024-09
    YearMonth nowPlus2Months = now.plusMonths(2);
    System.out.println(nowPlus2Months); // 2024-11
    System.out.println(nowPlus2Months.plusMonths(2)); // 2025-01
    System.out.println(nowPlus2Months.plusYears(2)); // 2026-11
    System.out.println(now.isAfter(nowPlus2Months)); // false

    // Se instancia un rango de mes-año [09/2024, 01/2025] donde el salto es de 1 mes
    YearMonthRange ymRange = new YearMonthRange(
        YearMonth.of(2024, 9), YearMonth.of(2025, 1));
    for(YearMonth aux : ymRange) {
        System.out.println(aux);
    }
    // 2024-09
    // 2024-10
    // 2024-11
    // 2024-12
    // 2025-01

    // Se instancia un rango de mes-año [09/2024, 03/2025] donde el salto es de 6 meses
    YearMonthRange ymRange6Months = new YearMonthRange(
        YearMonth.of(2024, 9), YearMonth.of(2025, 3),
        aYearMonth → aYearMonth.plusMonths(6));
    for(YearMonth aux : ymRange6Months) {
        System.out.println(aux);
    }
    // 2024-09
    // 2025-03

    // Se instancia un rango de mes-año [09/2024, 03/2027] donde el salto es de 2 años
    YearMonthRange ymRange2Years = new YearMonthRange(
        YearMonth.of(2024, 9), YearMonth.of(2027, 3),
        aYearMonth → aYearMonth.plusYears(2));
    for(YearMonth aux : ymRange2Years) {
        System.out.println(aux);
    }
    // 2024-09
    // 2026-09
}
```

Ejercicio 2 (3.5 puntos)

Se desea modelar un **gestor de artículos para su lectura**. Existen **tres tipos de artículos: RSS, PODCAST y YOUTUBE**. Un artículo se compone de un **tipo** y un **título**.

El gestor de artículos debe contar con un **método para agregar un artículo** y obtener la instancia correspondiente. Por defecto los artículos se instancian como “no leídos”. El artículo debe contar con un **método para marcarlo como leído**.

Para consultar los artículos agregados existen dos formas:

- **getArticlesCopy()** retorna los artículos **ordenado por tipo** donde primero aparecerán los de tipo PODCAST, luego YOUTUBE y por último RSS y, a igualdad de tipo, **desempata alfabéticamente por título**
- **getReversedArticlesCopy()** retorna los artículos **ordenado por tipo** donde primero aparecerán los de tipo RSS, luego YOUTUBE y por último PODCAST y, a igualdad de tipo, **desempata descendente por título**

Para consultar el primer artículo no leído (RESPETANDO EL ORDEN DE INSERCIÓN) existen dos formas:

- **firstUnread()** retorna el primer artículo no leído de cualquier tipo
- **firstUnread(tipoArticulo)** retorna el primer artículo no leído del tipo indicado por parámetro

En ambos casos si no existe al menos un artículo no leído que cumpla con lo pedido se arroja un error.

Implementar FeedType, Article, Feeder y todo lo necesario para que, con el siguiente programa de prueba, se obtenga la salida indicada en los comentarios:

```
public static void main(String[] args) {
    Feeder feeder = new Feeder();

    // Se agrega un artículo
    Article article1 = feeder.addArticle(FeedType.RSS, "What's New in Java 23 in 2 Minutes");
    System.out.println(article1); // RSS What's New in Java 23 in 2 Minutes is unread

    article1.read(); // Se marca al artículo como leído
    System.out.println(article1); // RSS What's New in Java 23 in 2 Minutes is read

    feeder.addArticle(FeedType.PODCAST, "Episode 31 Sequenced Collections");
    feeder.addArticle(FeedType.RSS, "JVM Language Summit and Java 23!");
    feeder.addArticle(FeedType.YOUTUBE, "Java 23 - Launch Stream");
    feeder.addArticle(FeedType.YOUTUBE, "Modern Java in Action");

    // Se obtienen los artículos con el siguiente orden:
    // Primero los Podcast, luego Youtube y por último RSS
    // y a igualdad de tipo de artículo desempata alfabéticamente por título
    for(Article article : feeder.getArticlesCopy()) {
        System.out.println(article);
    }
    // PODCAST Episode 31 Sequenced Collections is unread
    // YOUTUBE Java 23 - Launch Stream is unread
    // YOUTUBE Modern Java in Action is unread
    // RSS JVM Language Summit and Java 23! is unread
    // RSS What's New in Java 23 in 2 Minutes is read

    // Se obtienen los artículos con el siguiente orden:
    // Primero los RSS, luego Youtube y por último Podcast
    // y a igualdad de tipo de artículo desempata descendente por título
    for(Article article : feeder.getReversedArticlesCopy()) {
        System.out.println(article);
    }
}
```

```
// RSS What's New in Java 23 in 2 Minutes is read
// RSS JVM Language Summit and Java 23! is unread
// YOUTUBE Modern Java in Action is unread
// YOUTUBE Java 23 - Launch Stream is unread
// PODCAST Episode 31 Sequenced Collections is unread

// El primer artículo no leído de cualquier tipo (respetando el orden de inserción)
System.out.println(feeder.firstUnread());
// PODCAST Episode 31 Sequenced Collections is unread
feeder.firstUnread().read();
System.out.println(feeder.firstUnread());
// RSS JVM Language Summit and Java 23! is unread

// El primer artículo no leído de tipo RSS (respetando el orden de inserción)
System.out.println(feeder.firstUnread(FeedType.RSS));
// RSS JVM Language Summit and Java 23! is unread

System.out.println(feeder.firstUnread(FeedType.YOUTUBE));
// YOUTUBE Java 23 - Launch Stream is unread

try {
    // Se arroja un error porque no existen artículos no leídos de tipo Podcast
    feeder.firstUnread(FeedType.PODCAST);
} catch (IllegalArgumentException ex) {
    System.out.println("Error"); // Error
}
}
```

Ejercicio 3 (3.5 puntos)

Se desea modelar un sistema para la **administración de los alquileres de vehículos de una agencia** pudiendo alquilar un vehículo en una locación y fecha, llevar registro de los kilómetros recorridos y consultar si el alquiler puede finalizar en una locación de devolución y fecha determinada.

Ya cuenta con la clase RentalCentral (que NO puede modificarse) que corresponde a la central de alquileres y ofrece el comportamiento **para conocer, a partir del nombre, si una locación** (ya sea de alquiler o de devolución) **es aceptada** por la compañía:

```
public class RentalCentral {
    private String[] rentalLocations;

    public RentalCentral(String[] rentalLocations) {
        setRentalLocations(rentalLocations);
    }

    public boolean belongsIn(String rentalLocation) {
        for(String rL : rentalLocations) {
            if(rL.equals(rentalLocation)) {
                return true;
            }
        }
        return false;
    }

    public void setRentalLocations(String[] rentalLocations) {
        this.rentalLocations = rentalLocations;
    }
}
```

Para **instanciar un alquiler** se debe indicar una **locación de alquiler** (String) y una **fecha de alquiler** (Para ello deberá utilizar la clase inmutable **java.time.LocalDate** que modela una fecha -ya existe, no tiene que implementarla-). **La locación debe ser aceptada por la compañía**, de lo contrario se arroja un error. El alquiler **inicia con 0 kms** recorridos.

El alquiler debe contar con dos métodos:

- **addKms**: para registrar la cantidad de kms recorridos, a partir de un **entero**
- **canDrop**: para conocer si el alquiler puede finalizar, a partir de una **locación de devolución** (String) y una **fecha de devolución** (java.time.LocalDate)

Existen tres tipos de alquileres:

- Uno que puede finalizar si la **locación de devolución es aceptada** por la compañía y la **fecha de devolución es posterior** a la fecha del alquiler
- Uno que puede finalizar si la **locación de devolución coincide con la del alquiler** y la **fecha de devolución es posterior** a la fecha del alquiler y **NO supera** una **cantidad máxima de días** (que se indica al momento de instanciar el alquiler)
- Uno que sólo puede registrar una cantidad de kms recorridos siempre y cuando **NO supere una cantidad máxima de kms** (que se indica al momento de instanciar el alquiler) y que puede finalizar si la **locación de devolución es aceptada** por la compañía y la **fecha de devolución es posterior** a la fecha del alquiler y **SÍ supera** una **cantidad mínima de días** (que se indica al momento de instanciar el alquiler)

Para simplificar el ejercicio, se asume que los valores enteros de cantidad máxima de días, cantidad mínima de días como así también la cantidad de kms recorridos y la cantidad máxima de kms son todos valores positivos y no corresponde validarlo.

Completar losA.....B.....C....., e implementar todo lo necesario para que, con el siguiente programa de prueba, se obtenga la salida indicada en los comentarios:

```
public static void main(String[] args) {
    LocalDate now = LocalDate.of(2024, 9, 19); // Ejemplos de uso de java.time.LocalDate
    System.out.println(now); // 2024-09-19
    LocalDate nowPlus2Days = now.plusDays(2);
    System.out.println(nowPlus2Days); // 2024-09-21
    System.out.println(now.isBefore(nowPlus2Days)); // true
    System.out.println(nowPlus2Days.isAfter(now)); // true

    // Se instancia una central con dos locaciones conocidas
    String[] rentalLocations = new String[]{"Rotorua", "Paihia"};
    RentalCentral central = new RentalCentral(rentalLocations);

    // Se instancia un alquiler desde la locación Rotorua el 19/09/2024
    // donde se verifica que la locación de alquiler es conocida
    // y el alquiler podrá finalizarse si:
    //     la locación de devolución es conocida
    //     y la fecha de devolución es posterior a la de alquiler
    ..... rental = new .....;
    System.out.println(rental); // Rental from Rotorua on 2024-09-19 has 0 kms
    rental.addKms(100); // Se le adicionan 100kms al alquiler
    System.out.println(rental); // Rental from Rotorua on 2024-09-19 has 100 kms
    // Se consulta si este alquiler puede finalizar en la locación Paihia el 23/09/2024
    System.out.println(rental.canDrop("Paihia", LocalDate.of(2024, 9, 23))); // true
    System.out.println(rental.canDrop("Rotorua", LocalDate.of(2023, 1, 1))); // false
    System.out.println(rental.canDrop("Queenstown", LocalDate.of(2024, 9, 23))); // false
}
```

```

// Se instancia un alquiler desde la locación Rotorua el 19/09/2024
// con máxima cantidad de días (3)
// donde se verifica que la locación de alquiler es conocida
// y el alquiler podrá finalizarse si:
//     la locación de devolución coincide con la de alquiler
//     y la fecha de devolución es posterior a la de alquiler
//     y no supera la máxima cantidad de días
..... sameDropRental = new .....;
System.out.println(sameDropRental);
// Same Drop Rental from Rotorua on 2024-09-19 has 0 kms for up to 3 days
System.out.println(sameDropRental.canDrop("Rotorua", LocalDate.of(2024, 9, 22))); // true
System.out.println(sameDropRental.canDrop("Rotorua", LocalDate.of(2024, 9, 23))); // false
System.out.println(sameDropRental.canDrop("Paiahia", LocalDate.of(2024, 9, 22))); // false
sameDropRental.addKms(200);
System.out.println(sameDropRental);
// Same Drop Rental from Rotorua on 2024-09-19 has 200 kms for up to 3 days

// Se instancia un alquiler desde la locación Rotorua el 19/09/2024
// con mínima cantidad de días (5) y máxima cantidad de kms (300)
// donde se verifica que la locación de alquiler es conocida
// y que la adición de kms no supere el máximo indicado
// y el alquiler podrá finalizarse si:
//     la locación de devolución es conocida
//     y la fecha de devolución es posterior a la de alquiler
//     y supera la mínima cantidad de días
..... maxKmRental = new .....;
System.out.println(maxKmRental.canDrop("Rotorua", LocalDate.of(2024, 9, 20))); // false
System.out.println(maxKmRental.canDrop("Rotorua", LocalDate.of(2025, 1, 1))); // true
System.out.println(maxKmRental.canDrop("Paiahia", LocalDate.of(2024, 9, 24))); // true
maxKmRental.addKms(200);
System.out.println(maxKmRental);
// Max Rental from Rotorua on 2024-09-19 has 200 kms, at least 5 days and up to 300 kms
maxKmRental.addKms(100);
System.out.println(maxKmRental);
// Max Rental from Rotorua on 2024-09-19 has 300 kms, at least 5 days and up to 300 kms

// Falla pues de añadirse los kms serían 301 y supera los 300
try {
    maxKmRental.addKms(1);
} catch (RentalException ex) {
    System.out.println(ex.getMessage()); // Reached Max Kms Limit
}

// Se actualizan las locaciones conocidas por la central
central.setRentalLocations(new String[]{"Paiahia"});

// Falla pues se instancia un alquiler desde la locación Rotorua el 19/09/2024
// y NO se verifica que la locación de alquiler es conocida
try {
    ..... wrongRental = new .....A.....;
} catch (RentalException ex) {
    System.out.println(ex.getMessage()); // Pickup Location Not Found
}

System.out.println(rental.canDrop("Rotorua", LocalDate.of(2024, 9, 23))); // false
System.out.println(sameDropRental.canDrop("Rotorua", LocalDate.of(2024, 9, 22))); // false
System.out.println(maxKmRental.canDrop("Rotorua", LocalDate.of(2025, 1, 1))); // false
}

```